Alma Mater Studiorum | Università di Bologna
Laurea in Ingegneria Informatica
A.A. 2020-2021 | Sessione I

Progettazione di Applicazioni Web

basate su **Rust** e **WebAssembly**

Relatore
Prof. Paolo Bellavista

Tesi di Laurea di
Cristian Davide Conte

STORIA DEL WEB

1945
Pubblicazione di “As We May Think” sulla rivista “The Atlantic”

1993
Nascita del Web 1.0: “The Read-Only Web”

2004
Nascita del Web 2.0: “The Dynamic Web”

2013
Presentazione ufficiale del progetto “asm.js”

2015
Annuncio del formato “WebAssembly”

WebAssembly
I.S.A.

Funzionamento basato su quello di una **Virtual Stack Machine**.
Operazioni possibili: 1. Aritmetiche
2. Controllo

Per le allocazioni dei dati utilizza un Array di Byte, diviso in porzioni di 64KiB chiamate “Page”, che può solo crescere e che viene chiamato “**Linear Memory**”

Codice diviso in **moduli** costituiti da un “header” e da una sequenza di sezioni che indicano all’**host environment** come trattare ogni modulo

WebAssembly
Host Environment

Si occupa del **caricamento dei moduli** WebAssembly e delle interazioni con l'esterno (File, I/O, Socket…)

Solitamente **è un browser Web** che utilizza una macchina virtuale in grado di eseguire istruzioni WebAssembly in un ambiente isolato

Si occupa di effettuare il **polyfill ad** "**asm.js**" nel caso in cui il formato WebAssembly non sia supportato

STORIA DI RUST

2006

L'ingegnere Graydon Hoare sviluppa una versione alpha di Rust come suo progetto personale

2010

Mozilla annuncia ufficialmente Rust Programming Language

2012

Viene rilasciato al pubblico il primo compilatore Rust

2015

Mozilla rilascia al pubblico la prima versione stabile di Rust

2016-2020

Rust diventa il "Most Loved Programming Language" su StackOverflow

Rust
Caratteristiche

Linguaggio di basso livello con **efficienza** e **memory safety** come colonne portanti

Promette le "**Zero Cost Abstractions**": soluzioni in *functional programming* ed in *imperative programming* sono compilate esattamente nello stesso Assembly

Non utilizza **alcun Garbage Collector** e garantisce un uso sicuro della memoria tramite regole di "**Ownership & Borrowing**"

Rust

Memory Safety Rules

Ownership

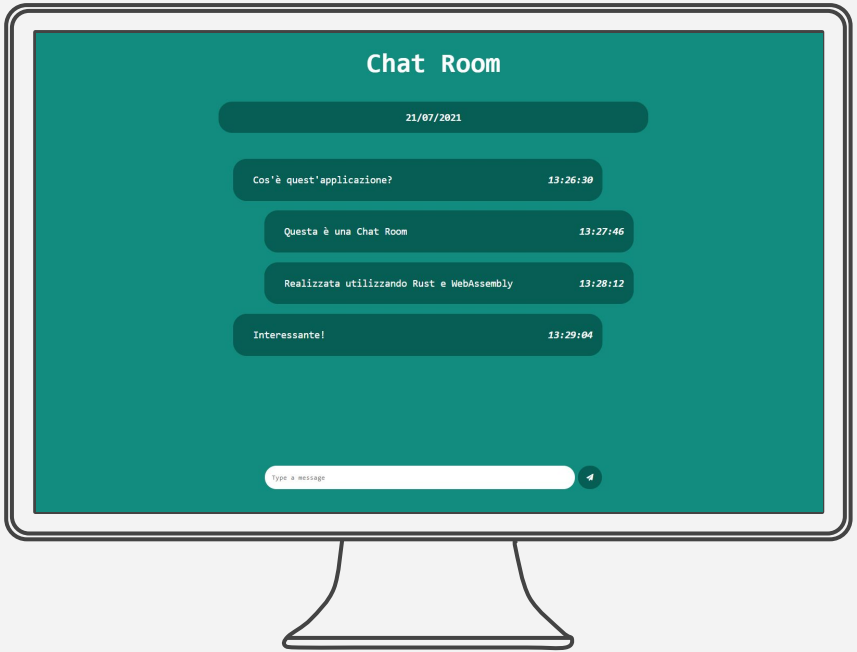
Ogni valore è contenuto all'interno di una variabile definita come "owner" di quel valore

Questa variabile è unica in ogni istante di tempo

Borrowing

Serve per "prendere in prestito" il valore di una variabile utilizzando un riferimento ad essa

Possono coesistere infiniti riferimenti immutabili od un unico riferimento mutabile



Applicazione Web in **Rust** e **WebAssembly**

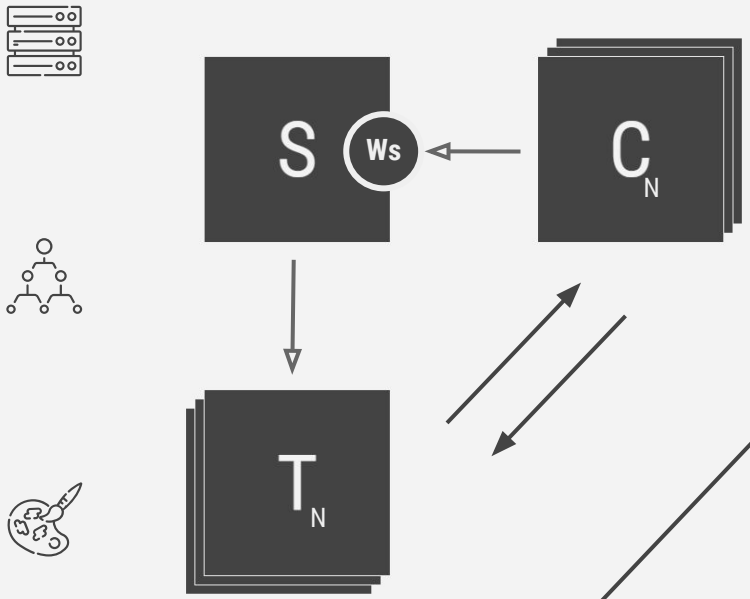
Chat Room

Chat Room
Architettura

Utilizzo di un **Server** in ascolto tramite **WebSocket** Rust per eventuali richieste di connessioni o invio messaggi, che tiene traccia del tempo associato a questi ultimi

Utilizzo di **Thread** Rust per servire concorrentemente le richieste e gestire il tempo di messaggi e date

Client frontend che utilizza la libreria ***web-sys*** e il framework ***Yew***, basato su eventi e componenti, per gestire le interazioni con l'utente e mostrare la cronologia dei messaggi



Libreria Rust

web-sys

- È una libreria che fornisce un'implementazione delle **WebAPI** standard all'interno del codice Rust
- Associata allo strumento di compilazione ***wasm-bindgen***, permette di compilare il codice Rust in formato WebAssembly
- **Sfrutta la** "**Linear Memory**" WASM per scambiare dati tra Rust/WebAssembly e ambiente host/Javascript: serializza i dati in sequenze di byte e li deserializza in oggetti Javascript/struct Rust
- Permette di utilizzare le funzioni Javascript che interagiscono con il DOM all'interno del codice Rust per la **manipolazione di documenti HTML**

Conclusioni

Uno sguardo al futuro

L'utilizzo di Rust e WebAssembly, attualmente esiguo, è in crescita e queste tecnologie hanno raggiunto una maturità tale da poterle utilizzare per la creazione di Applicazioni Web **efficienti**, **portabili** e di **scalabili**

Architetture Client/Server potrebbero diventare **Client-only** laddove si riuscisse a sfruttare la potenza di calcolo dei moderni dispositivi tramite WebAssembly, riducendo latenze e costi per le aziende

Wasmtime & **Wasmer**: due runtime per WASM, in sviluppo, che permetteranno di eseguire codice WebAssembly in ambienti host diversi dal browser uniformando il processo di sviluppo delle applicazioni

Grazie